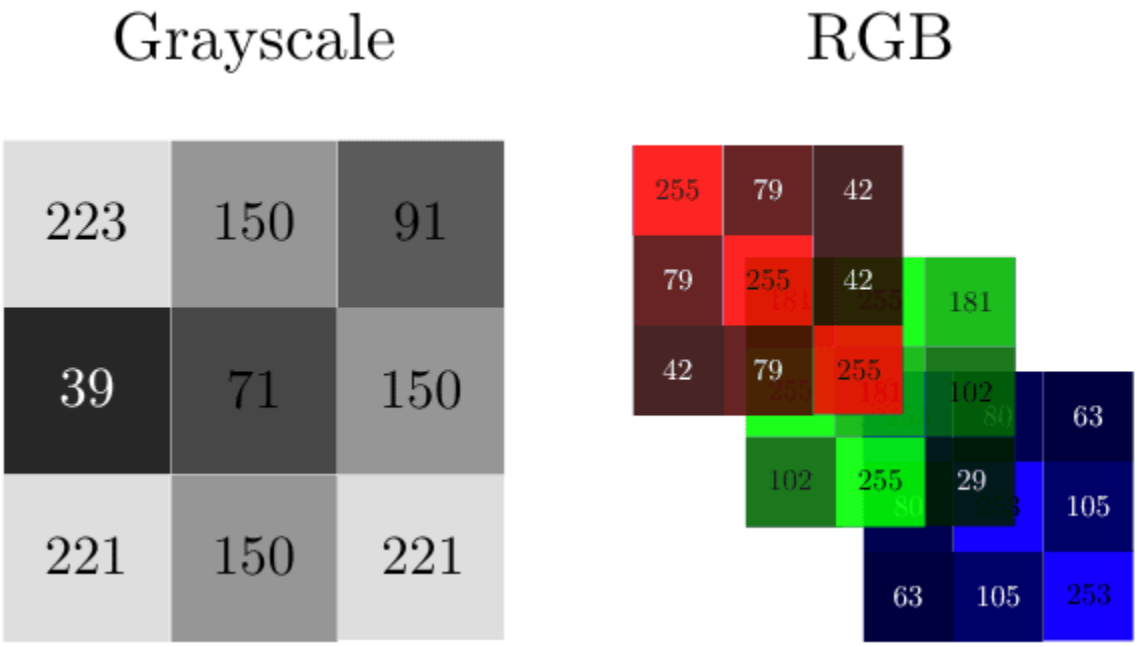
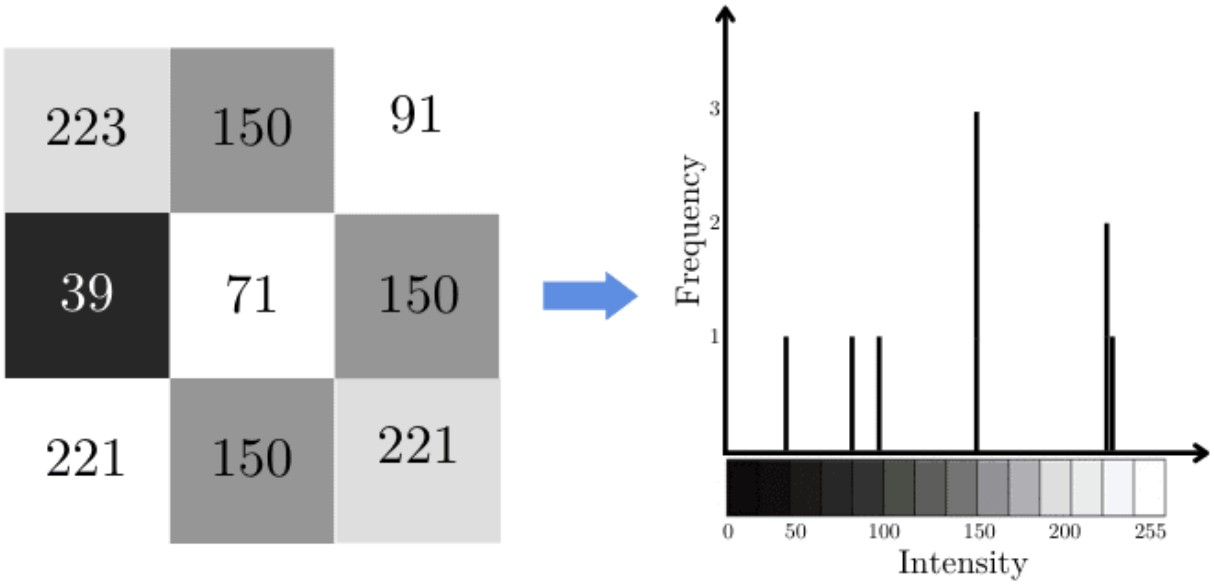
In the case of a grayscale image, this matrix will be made of numbers between 0 and 255. For RGB images, we'll have three matrices, one of each color channel. For instance:



Histogram:

histogram of an image as a 2D bar plot. The horizontal axis represents the pixel intensities. The vertical axis denotes the frequency of each intensity. To determine the histogram of an image, we need to count how many instances of each intensity we have.

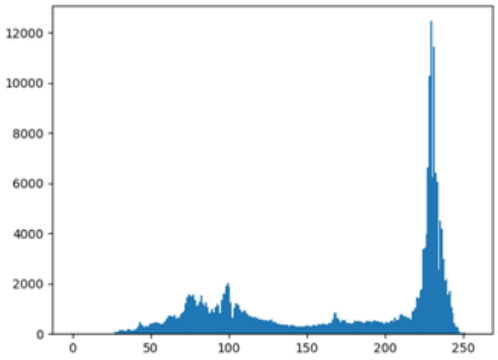


Applications:

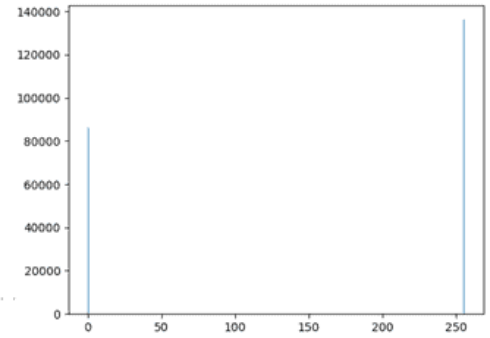
Histogram for Image Segmentation:

We can use histograms to define the threshold for image segmentation to isolate the background from an object.

For instance, if we want to isolate a rose in the following image from its background, we can start by analyzing its histogram. In this way, we can see that most of the background pixels are white or whiteish. This means that most of the background pixels are close to 255:

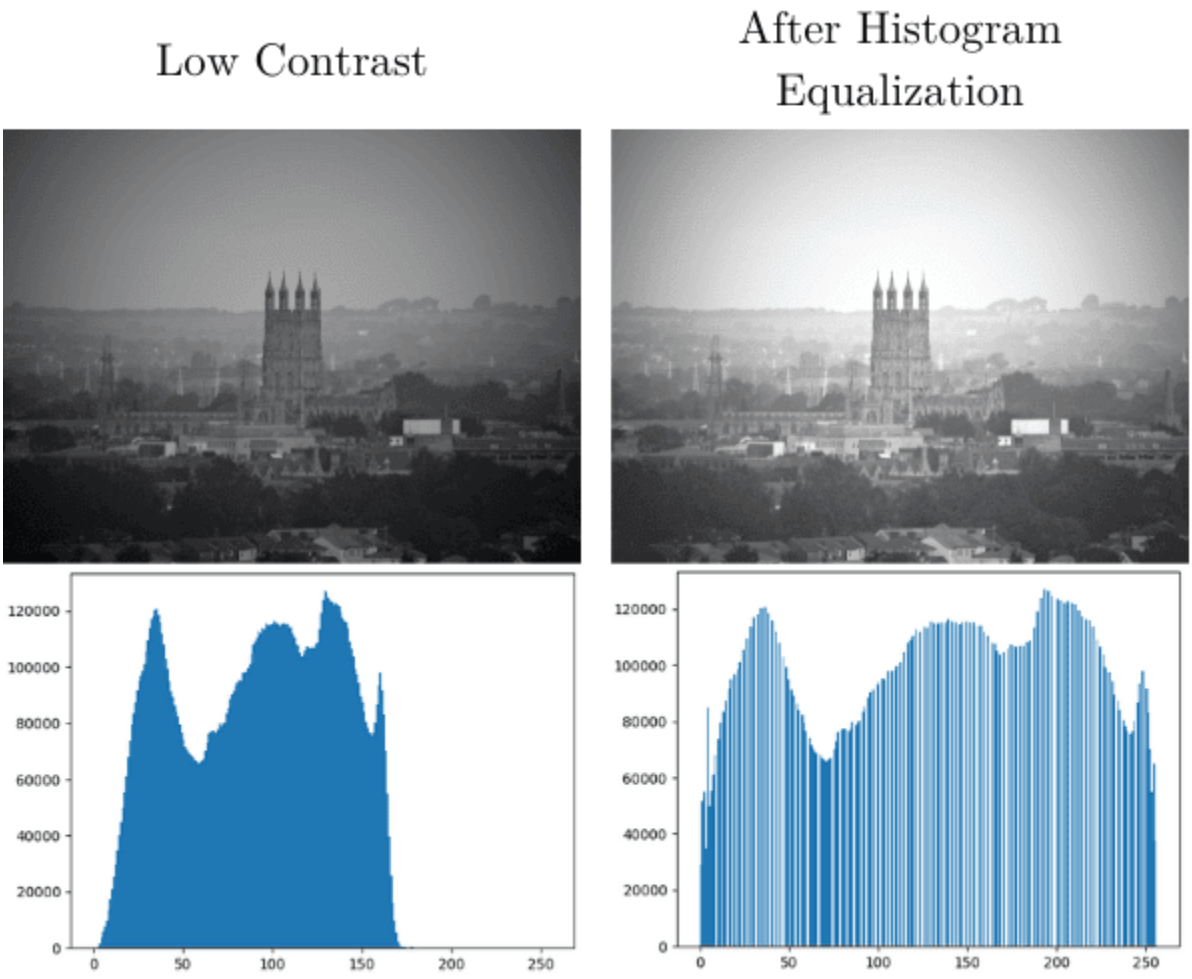


If we define our threshold as 156, and take every pixel > 156 to belong to the background, we'll get a binary image in which the rose is clearly separated:

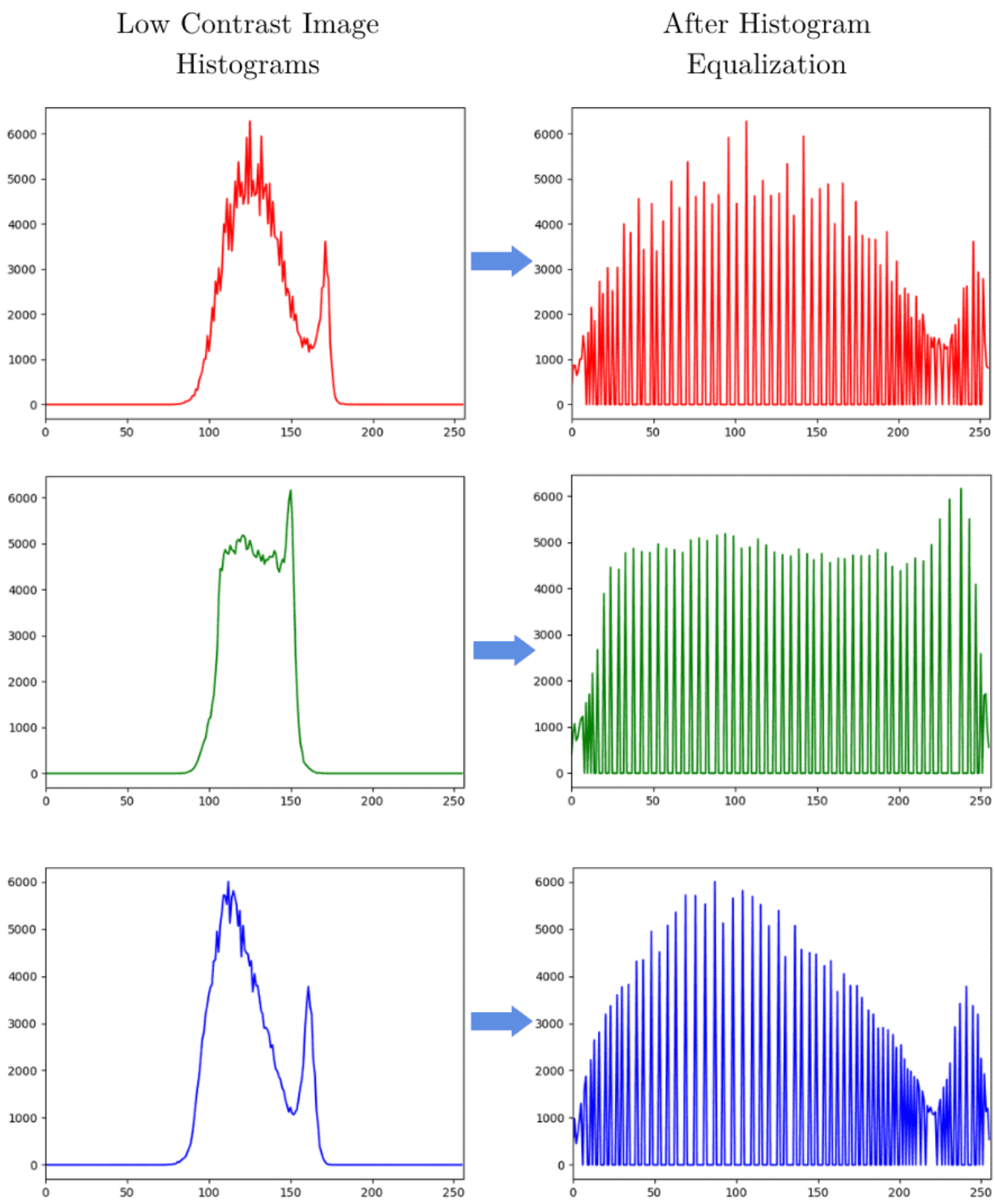


Histograms in Photography

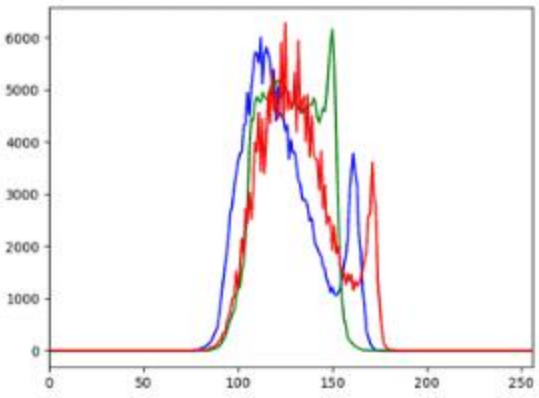
In photography, we use histograms to enhance pictures by changing some of their properties. This might help us get clearer pictures or even more beautiful photos.



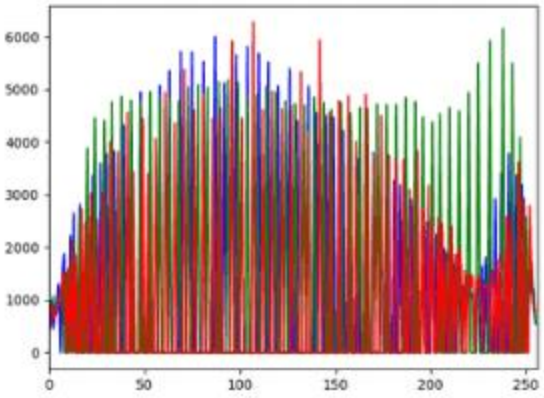
RGB Histograms



Low Contrast



After Histogram Equalization



CODE:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt


image = cv2.imread('image/test.jpg', cv2.IMREAD_GRAYSCALE)


histogram_cv = cv2.calcHist([image], [0], None, [256], [0, 256]).flatten()
```

- Function: cv2.calcHist() calculates the histogram of an image.
- Input: [image] — the image, passed as a list.
- Channel: [0] — the 0th channel (grayscale or blue channel in BGR).
- Mask: None — compute histogram for the entire image.

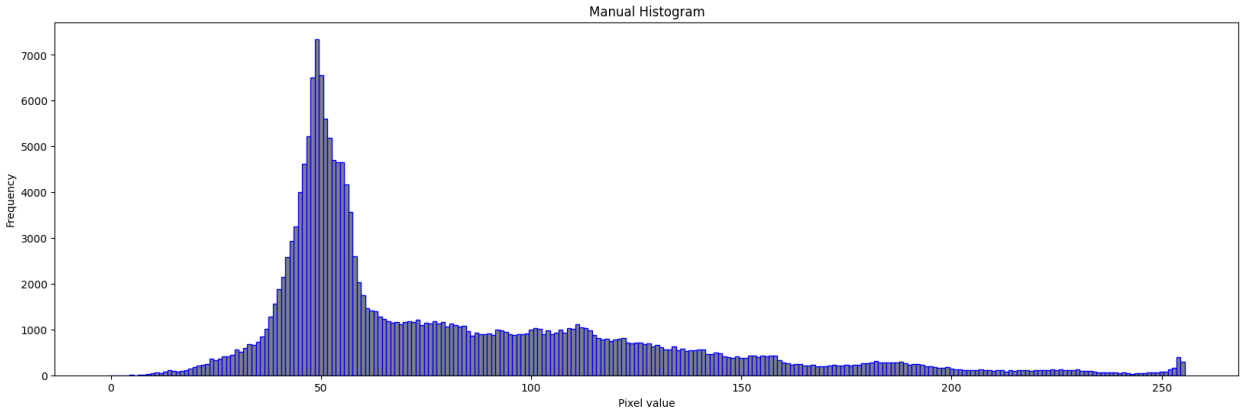
  This is the mask. It allows you to specify a region of interest (ROI) if you only want to calculate the histogram for part of the image.

  None means the histogram is computed for the entire image.

- Bins: [256] — number of bins (one for each intensity level from 0 to 255).
- Range: [0, 256] — intensity value range (inclusive of 0, exclusive of 256).
- Output: 2D array of shape (256, 1), showing count of pixels at each intensity.
- .flatten() converts it to a 1D array of shape (256,).

```
plt.figure(figsize=(20, 6))
plt.title("OpenCV Histogram")
plt.xlabel("Pixel value")
plt.ylabel("Frequency")
plt.bar(range(256), histogram_cv, width=1, edgecolor='red', align='center',
color='gray')
plt.show()
```

Histogram and Histogram Equalization

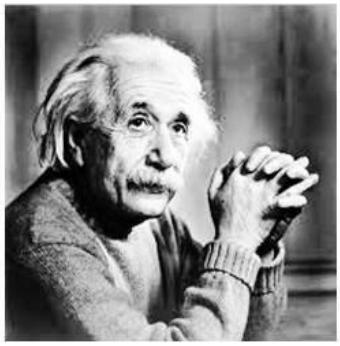


DO IT MANUALLY

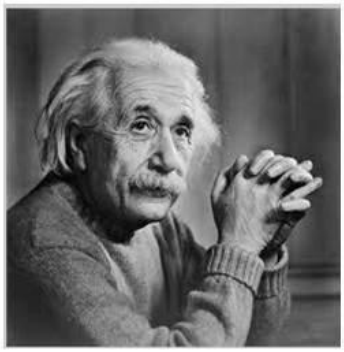
Histogram Equalization:

Histogram Equalization is a technique to adjust contrast levels and expand the intensity range in a digital image. It tries to enhance the image and helps image processing easier.

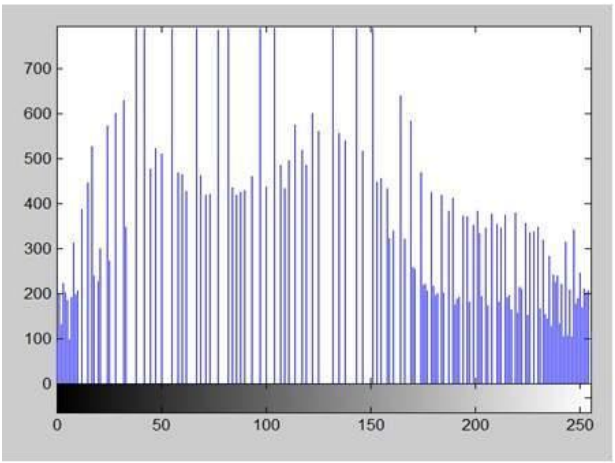
New Image



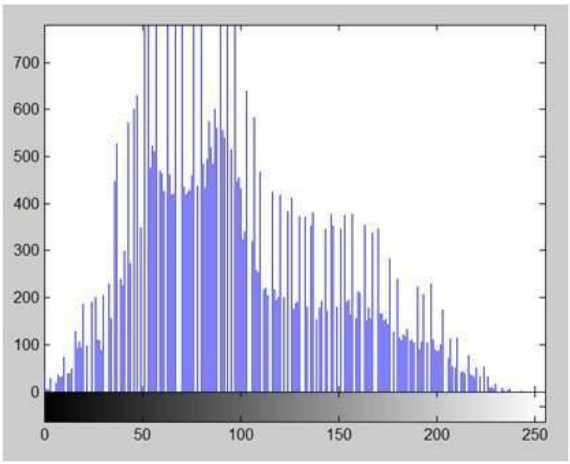
Old image



New Histogram



Old Histogram



1. Convert the input image to a grayscale image.
2. Calculate the frequency of occurrence of each pixel value.
3. Calculate Cumulative frequency of all pixel values.
4. Divide the cumulative frequencies by total number of pixels and multiply them by the maximum pixel value in the image.

0	0	0	1	1	1
2	2	3	2	2	7
3	0	7	4	7	5
3	4	4	2	4	5
2	7	6	7	7	5
6	5	3	6	6	0

Pixel value	Frequency	Cumulative freq	Normalized freq	New pixel value
0	5	5	5/36	7*5/36=1.x
1	3	8	8/36	7*8/36=2.x
2	6	14	14/36	7*14/36=3.x
3	4	18	18/36	7*18/36=4.x
4	4	22	22/36	7*22/36=4.x
5	4	26	26/36	7*26/36=5.x
6	4	30	30/36	7*30/36=6.x
7	6	36	36/36	7*36/36=7

0	0	0	1	1	1
2	2	3	2	2	7
3	0	7	4	7	5
3	4	4	2	4	5
2	7	6	7	7	5
6	5	3	6	6	0

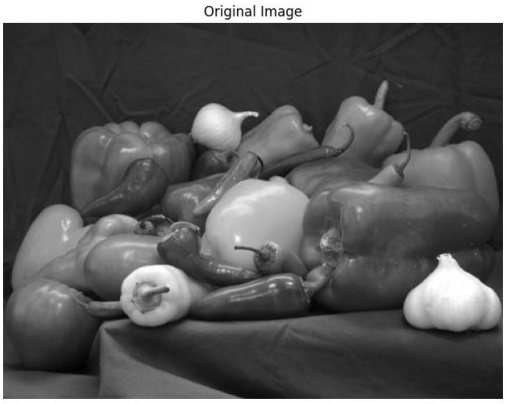
1	1	1	2	2	2
3	3	4	3	3	7
4	1	7	4	7	5
4	4	4	3	4	5
3	7	6	7	7	5
6	5	4	6	6	1

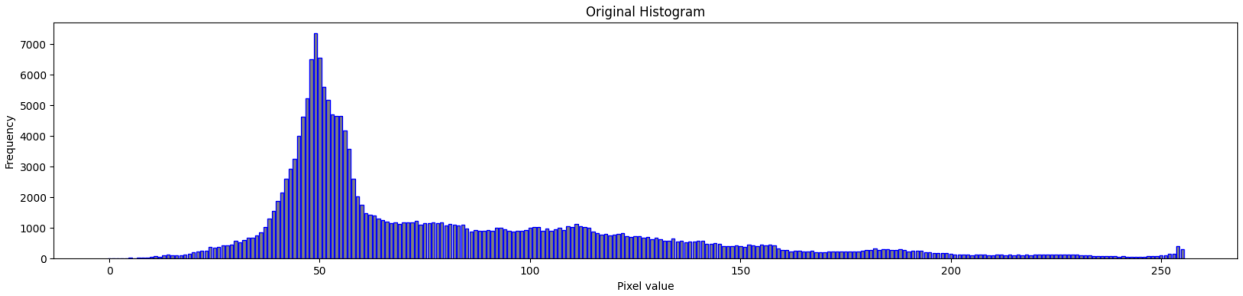
CODE:

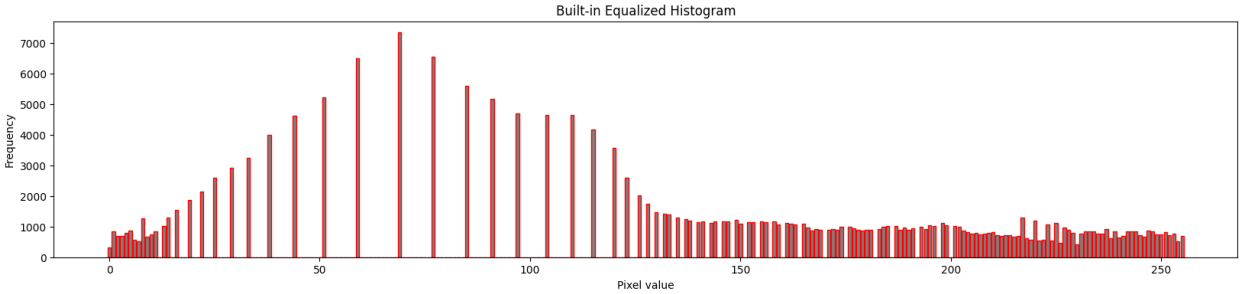
```
import cv2
import numpy as np
from matplotlib import pyplot as plt

equalized_image_builtin = cv2.equalizeHist(image)
```

Histogram and Histogram Equalization







ALSO DO IT MANUALLY